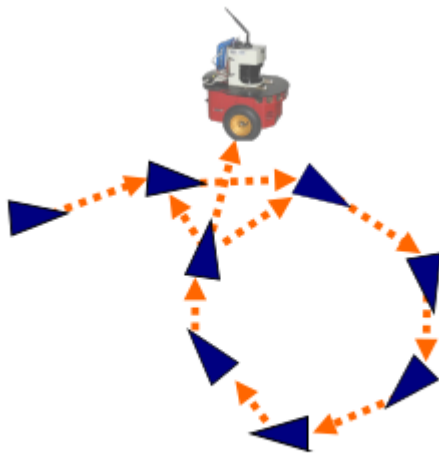


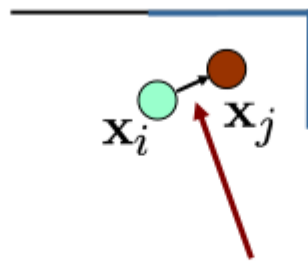
L16: Graph-based SLAM using Pose Graphs

- Alternative to Kalman Filter and Particle Filter
- Least Squares approach
 - Computing solution for overdetermined system
 - "More equations than unknowns"
 - Minimises sum of squared errors in equations
 - Standard approach to a large set of problems



► Robot pose ...► Constraint

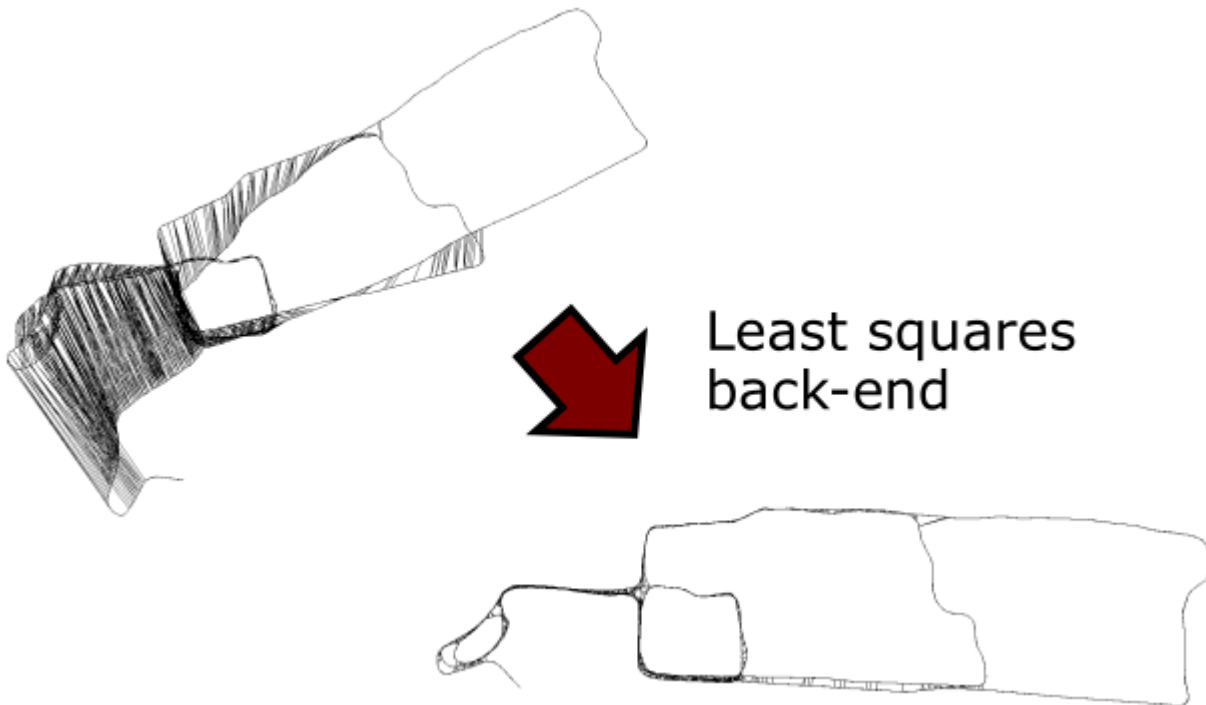
- Constraints connect robot poses while moving
- Constraints are inherently uncertain
- Observing previously seen areas generates constraints between non-successive poses
- **Graph** represents problem
- **Node**: Robot Pose at given Point in Time
 - $x = x_{1:n}$
- **Edge**: Spatial constraint between 2 nodes
 - robot moves from x_i to x_{i+1}
 - represents **odometry** measurements
 - Construct virtual measurement about position of x_j seen from x_i
 - via scan matching



**via scan
matching**

Edge represents the pose of x_j seen from x_i based on the **observation**

- Graph-based SLAM
 - **Graph Construction:** Build graph and
 - **Graph-Optimisation:** Find node configuration:
 - minimise error introduced by the constraints



Transformations

- x_i sees x_j if:
 - expressed as transformation
 - relative transformation
 - $X_I^{-1} X_j$
 - X_i : transformation of origin to x_i
 - X_i^{-1} : inverse transformation
- Homogeneous coordinates
 - Odometry edge (Computed given graph)
 - $(X_i^{-1} X_{i+1})$

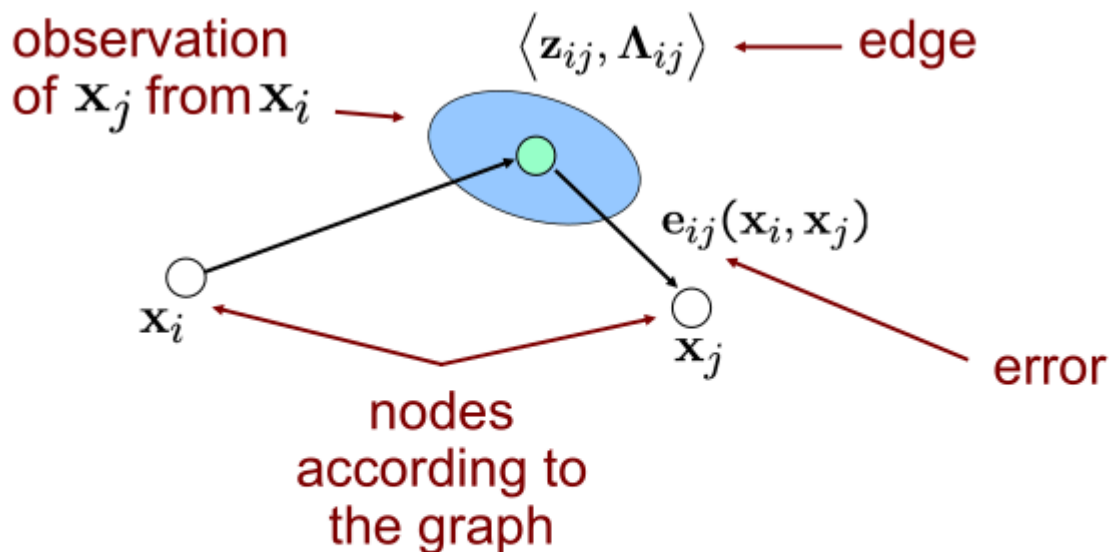
- Observation edge (Computed given the graph)

- $(X_i^{-1}X_j)$

- How node i sees node j

Edge Information Metrics

- Observation affected by noise
- Information matrix Λ_{ij} to encode uncertainty
 - for each edge
 - The bigger, the more impact of edge on optimisation



Goal: $x^* = \underset{x}{\operatorname{argmin}} \sum_{ij} e_{ij}^\top \Lambda_{ij} e_{ij}$

- Error Function for Least Squares Error Minimisation

$$x^* = \underset{x}{\operatorname{argmin}} \sum_k e_k^\top(x) \Lambda_k e_k(x)$$

- State Vector

$$x^T = (x_1^T \quad x_2^T \quad \dots \quad x_n^T)$$

- One Vector for each node of the graph
- Specify error function:

- for single constraint

$$e_{ij}(x_i, x_j) = \operatorname{t2v} \left(\underbrace{Z_{ij}^{-1}}_{\text{measurement}} \left(\underbrace{X_i^{-1}X_j}_{x_j \text{ referenced w.r.t. } x_i} \right) \right)$$

- Error as function of whole state vector

$$e_{ij}(\mathbf{x}) = \text{t2v}(Z_{ij}^{-1}(X_i^{-1}X_j))$$

- Error takes value of zero if
 - $Z_{ij} = (X_i^{-1}X_j)$

Gauss-Newton

1. Define error function
2. Linearise error function
3. Build linear system
4. Solve linear system
5. Iterate procedure until convergence

Linearising Error Function

- Approximate error function around initial guess $\mathbf{x}$
 - via Taylor expansion
- Derivative of Error function
 - $e_{ij}(\mathbf{x})$ depends only on x_i and x_j
 - not on all state variable
- Structure of Jacobian:
 - Only non-zero in rows corresponding to x_i and x_j

$$J_{ij} = \begin{pmatrix} \boxed{0 \dots 0} & \boxed{\underbrace{\frac{\partial e(\mathbf{x}_i)}{\partial \mathbf{x}_i}}_{A_{ij}}} & \boxed{0 \dots 0} & \boxed{\underbrace{\frac{\partial e(\mathbf{x}_j)}{\partial \mathbf{x}_j}}_{B_{ij}}} & \boxed{0 \dots 0} \end{pmatrix}$$

3x3 **3x3**

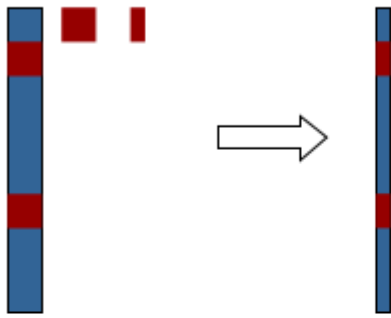
- Compute coefficient vector b and matrix H
 - using Jacobian

$$b^T = \sum_{ij} b_{ij}^T = \sum_{ij} e_{ij}^T \Lambda_{ij} J_{ij}$$

$$H = \sum_{ij} H_{ij} = \sum_{ij} J_{ij}^T \Lambda_{ij} J_{ij}$$

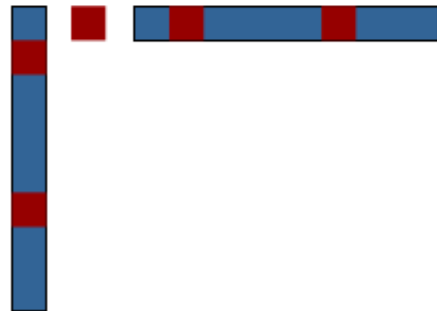
- Sparse structure of J_{ij} results in sparse structure of H
- Reflects on Adjacency matrix of the graph

$$\mathbf{b}_{ij} = \mathbf{J}_{ij}^\top \mathbf{\Lambda}_{ij} \mathbf{e}_{ij}$$

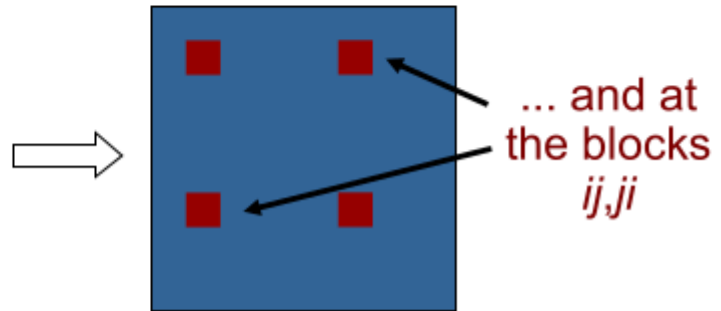


Non-zero only at $\mathbf{x}_i$ and $\mathbf{x}_j$

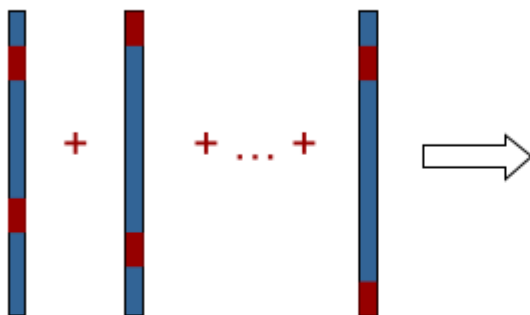
$$\mathbf{H}_{ij} = \mathbf{J}_{ij}^\top \mathbf{\Lambda}_{ij} \mathbf{J}_{ij}$$



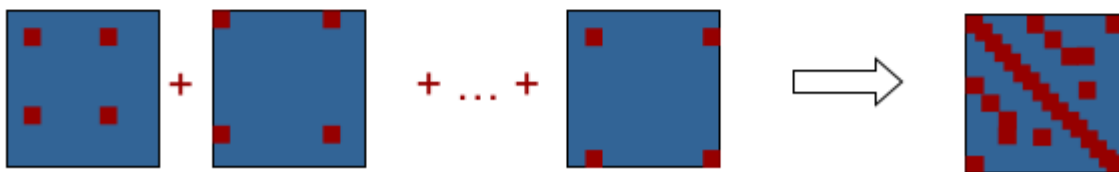
Non-zero on the main diagonal at $\mathbf{x}_i$ and $\mathbf{x}_j$



$$\mathbf{b} = \sum_{ij} \mathbf{b}_{ij}$$



$$\mathbf{H} = \sum_{ij} \mathbf{H}_{ij}$$



- H matrix for a single edge

$$\begin{aligned}
H_{ij} &= J_{ij}^T \Lambda_{ij} J_{ij} \\
&= \begin{bmatrix} \vdots \\ A_{ij}^T \\ \vdots \\ B_{ij}^T \\ \vdots \end{bmatrix} \Lambda_{ij} \begin{bmatrix} \dots & A_{ij} & \dots & B_{ij} & \dots \end{bmatrix} \\
&= \begin{bmatrix} A_{ij}^T \Lambda_{ij} A_{ij} & A_{ij}^T \Lambda_{ij} B_{ij} \\ B_{ij}^T \Lambda_{ij} A_{ij} & B_{ij}^T \Lambda_{ij} B_{ij} \end{bmatrix}
\end{aligned}$$

- Edge ij only contribute to
 - i^{th} and j^{th} block of b_{ij}
 - to blocks ii, jj, ij, ji of H_{ij}
- H is sparse but b is not
- Linear system computable by summing up contribution of each edge
- Linear system is sparse
- Efficient solvers used
 - Sparse Cholesky decomposition
 - Conjugate gradients

Linear System

- Vector of states increments:

$$\Delta x^T = [\Delta x_1^T \quad \Delta x_2^T \quad \dots \Delta x_n^T]$$

- Coefficient vector:

$$b^T = [\bar{b}_1^T \quad \bar{b}_2^T \quad \dots \bar{b}_n^T]$$

- Normal equation matrix:

$$H = \begin{bmatrix} \bar{H}^{11} & \bar{H}^{12} & \dots & \bar{H}^{1n} \\ \bar{H}^{21} & \bar{H}^{22} & \dots & \bar{H}^{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \bar{H}^{n1} & \bar{H}^{n2} & \dots & \bar{H}^{nn} \end{bmatrix}$$

Building Linear System

- Compute error
 - $e_{ij}(x) = t_2 v(Z_{ij}^{-1}(X_i^{-1} X_j))$
- Compute blocks of Jacobian

- $A_{ij} = \frac{\partial e(x_i, x_j)}{\partial x_i}$
- $B_{ij} = \frac{\partial e(x_i, x_j)}{\partial x_j}$
- Update coefficient vector:
 - $\bar{b}_i^T + = e_{ij}^T \Lambda_{ij} A_{ij}$
 - $\bar{b}_j^T + = e_{ij}^T \Lambda_{ij} B_{ij}$
- Update Normal Equation matrix:
 - $\bar{H}^{ii} + = A_{ij}^T \Lambda_{ij} A_{ij}$
 - $\bar{H}^{ij} + = A_{ij}^T \Lambda_{ij} B_{ij}$
 - $\bar{H}^{ji} + = B_{ij}^T \Lambda_{ij} A_{ij}$
 - $\bar{H}^{jj} + = B_{ij}^T \Lambda_{ij} B_{ij}$

1: **optimize(x):**

2: while (!converged)

3: (**H**, **b**) = buildLinearSystem(**x**)

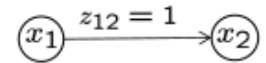
4: **Δx** = solveSparse(**HΔx** = -**b**)

5: **x** = **x** + **Δx**

6: end

7: return **x**

Trivial 1D Example



■ Two nodes and one observation

$$\begin{aligned} \mathbf{x} &= (x_1 \ x_2)^\top = (0 \ 0)^\top \\ z_{12} &= 1 \\ \Lambda &= 2 \\ \mathbf{e}_{12} &= z_{12} - (x_2 - x_1) = 1 - (0 - 0) = 1 \\ \mathbf{J}_{12} &= (1 \ -1) \\ \mathbf{b}_{12}^\top &= \mathbf{e}_{12}^\top \Lambda_{12} \mathbf{J}_{12} = (2 \ -2) \\ \mathbf{H}_{12} &= \mathbf{J}_{12}^\top \Lambda \mathbf{J}_{12} = \begin{pmatrix} 2 & -2 \\ -2 & 2 \end{pmatrix} \\ \Delta \mathbf{x} &= -\mathbf{H}_{12}^{-1} \mathbf{b}_{12} \quad \text{BUT } \det(\mathbf{H}) = 0 \text{ ???} \end{aligned}$$

- Constraint specifies **relative constraint**
 - Any poses fine, as long as relative coordinates fit
- One node needs to be fixed

$$\begin{aligned} \mathbf{H} &= \begin{pmatrix} 2 & -2 \\ -2 & 2 \end{pmatrix} + \boxed{\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}} \quad \text{constraint that sets } \mathbf{dx}_1 = \mathbf{0} \\ \Delta \mathbf{x} &= -\mathbf{H}^{-1} \mathbf{b}_{12} \\ \Delta \mathbf{x} &= (0 \ 1)^\top \end{aligned}$$

Role of prior Node / Gauge Freedom

- Matrix $\mathbf{H}$ has not full rank (after adding constraints)
- Global frame had to be fixed
- Fixing global reference frame strongly related to prior $p(x_0)$
- Gaussian estimate about x_0 results in an additional constraint
- E.g., first pose is the origin:
 - $e(x_0) = 2tv(X_0)$

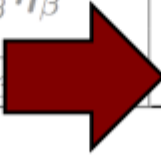
Fixing a Subset of Variables

- Assume that the value of certain variable during the optimisation is known a priori

- Optimise all others - keep these fixed
- Variable not optimised: disappear from linear system
- Construct full system
- Suppress rows and columns corresponding to variables to fix

$$p(\boldsymbol{\alpha}, \boldsymbol{\beta}) = \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu}_\alpha \\ \boldsymbol{\mu}_\beta \end{bmatrix}, \begin{bmatrix} \Sigma_{\alpha\alpha} & \Sigma_{\alpha\beta} \\ \Sigma_{\beta\alpha} & \Sigma_{\beta\beta} \end{bmatrix}\right) = \mathcal{N}^{-1}\left(\begin{bmatrix} \boldsymbol{\eta}_\alpha \\ \boldsymbol{\eta}_\beta \end{bmatrix}, \begin{bmatrix} \Lambda_{\alpha\alpha} & \Lambda_{\alpha\beta} \\ \Lambda_{\beta\alpha} & \Lambda_{\beta\beta} \end{bmatrix}\right)$$

	MARGINALIZATION	CONDITIONING
	$p(\boldsymbol{\alpha}) = \int p(\boldsymbol{\alpha}, \boldsymbol{\beta}) d\boldsymbol{\beta}$	$p(\boldsymbol{\alpha} \boldsymbol{\beta}) = p(\boldsymbol{\alpha}, \boldsymbol{\beta}) / p(\boldsymbol{\beta})$
COV. FORM	$\boldsymbol{\mu} = \boldsymbol{\mu}_\alpha$ $\Sigma = \Sigma_{\alpha\alpha}$	$\boldsymbol{\mu}' = \boldsymbol{\mu}_\alpha + \Sigma_{\alpha\beta} \Sigma_{\beta\beta}^{-1} (\boldsymbol{\beta} - \boldsymbol{\mu}_\beta)$ $\Sigma' = \Sigma_{\alpha\alpha} - \Sigma_{\alpha\beta} \Sigma_{\beta\beta}^{-1} \Sigma_{\beta\alpha}$
INFO. FORM	$\boldsymbol{\eta} = \boldsymbol{\eta}_\alpha - \Lambda_{\alpha\beta} \Lambda_{\beta\beta}^{-1} \boldsymbol{\eta}_\beta$ $\Lambda = \Lambda_{\alpha\alpha} - \Lambda_{\alpha\beta} \Lambda_{\beta\beta}^{-1} \Lambda_{\beta\alpha}$	$\boldsymbol{\eta}' = \boldsymbol{\eta}_\alpha - \Lambda_{\alpha\beta} \boldsymbol{\beta}$ $\Lambda' = \Lambda_{\alpha\alpha}$



Copyright © 2011, John Wiley & Sons, Inc.

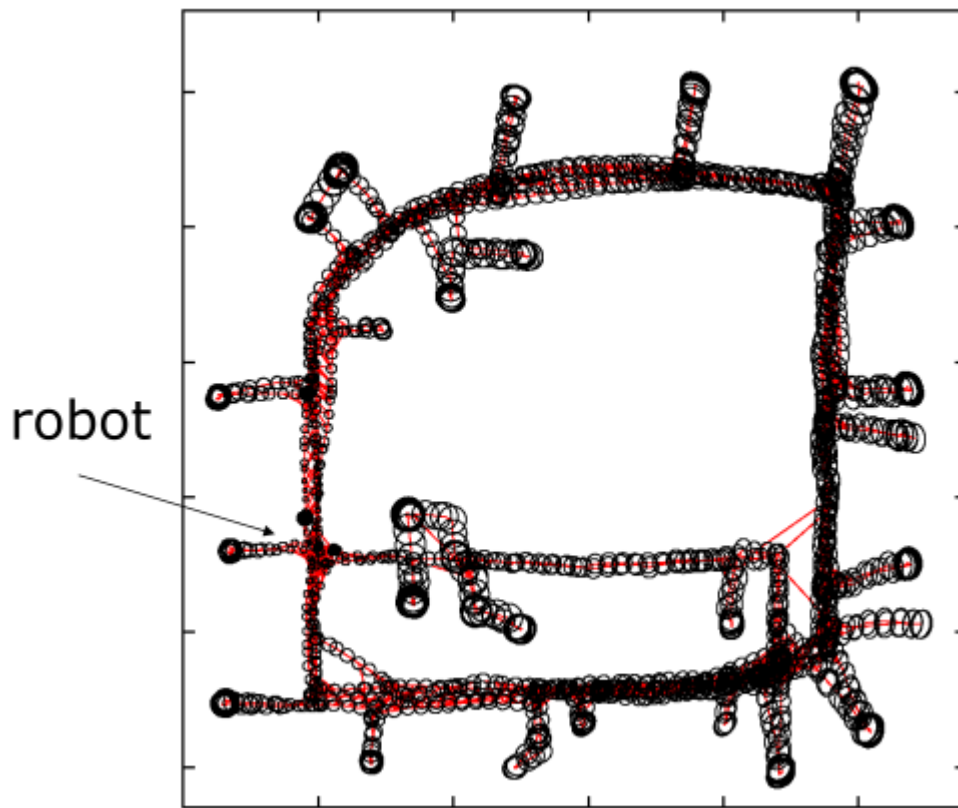
Uncertainty

- H represents the information matrix given the linearisation point
- Inverting H gives dense covariance matrix
- Diagonal blocks of covariance matrix represent the uncertainties of corresponding variables

Relative Uncertainty

- To determine the relative uncertainty between x_i and x_j :
 - Construct full matrix H
 - Suppress rows and columns of x_i
 - i.e. fix this variable
 - Compute block j, j of the inverse
 - This block will contain the covariance matrix of x_j given x_i (which has been fixed)

Example



- Gold standard solutions for optimising pose graphs and subsequently compute the environment map
- Note: good outlier handling is important (next lecture)